Tensor Network Simulation of the Heisenberg Model on Heavy-Hex Lattices

Paolo D'Alberto*
Advanced Micro Devices, Inc.
paolo.dalberto@amd.com

July 23, 2026

Abstract

We extend CppSim, a high-performance C++/HIP tensor network simulator, to the heavy-hex lattice geometry used in superconducting quantum processors. The heavy-hex graph — a honeycomb of degree-2 and degree-3 sites with a natural 3-color gate schedule — is implemented as a drop-in grid class with no changes to the gate application or belief propagation (BP) kernels. We simulate the isotropic Heisenberg model $H = J\sum_{\langle ij\rangle}(\mathbf{X}_i\mathbf{X}_j + \mathbf{Y}_i\mathbf{Y}_j + \mathbf{Z}_i\mathbf{Z}_j)$ in the Heisenberg picture using Pauli transfer matrices, and study the autocorrelation $C(t) = \langle Z_c(t)Z_c(0)\rangle$ and operator lightcone on the `heavy_hex_3x3` graph (68 sites, 76 bonds). We demonstrate chi convergence at $\chi = 200$ ($|\text{error}| < 10^{-4}$, reference $\chi = 430$) on a 32 GB GPU, and show a $\sim$10$\times$ speedup over a Julia/AMDGPU.jl reference at saturated bond dimension.

1 Introduction

Heavy-hex is the lattice topology used in superconducting quantum processors [1]. Each qubit site has degree 2 or 3, and the graph admits a natural 3-coloring of its bonds, enabling parallel gate application within each color class.

Classical tensor network simulation of quantum circuits on heavy-hex graphs serves two purposes: (i) validation of quantum computations for small system sizes where exact simulation is feasible [2, 3, 4, 5], and (ii) computation of noise-free reference values for quantum error mitigation protocols such as the multi-product formula (MPF) [6].

In this paper we show that CppSim [7], originally developed for rectangular 2D Ising dynamics, extends naturally to heavy-hex topology with minimal code changes. The hot-path kernels — gate application, BP, workspace management — are topology-agnostic and require no modification. Only the grid construction and Hamiltonian gate matrix need to be defined for the new geometry.

2 Heavy-Hex Lattice and Hamiltonian

We describe the graph topology, its implementation in CppSim, and the Hamiltonian gate matrix used for time evolution.

*Advanced Micro Devices, Inc. (AMD). paolo.dalberto@amd.com. This work was developed in active collaboration with Claude (Anthropic)

2.1 Graph structure

The `heavy_hex`$_{n_x} \times n_y$ graph has:

- $(n_x + 1)(n_y + 1)$ *junction* sites (degree 3)
- remaining *arm* sites (degree 2) connecting adjacent junctions
- $2n_xn_y + n_x + n_y$ bonds total

For $n_x = n_y = 3$: 16 junction sites, 52 arm sites, 76 bonds. The graph is bipartite and 3-edge-colorable — bonds partition into 3 color classes of sizes 25, 25, 26, corresponding to the natural gate layers of the hardware gate schedule.

Site tensors have shapes $[\chi^2, D]$ (arm) and $[\chi^3, D]$ (junction), where $D = 4$ is the Pauli basis dimension $\{I, X, Y, Z\}$. The memory cost is dominated by junction sites: $16 \times \chi^3 \times D \times 4\,\mathrm{B}$ (float32).

2.2 Implementation

The `FlatHeavyHexGrid` class inherits from `Grid` and populates the same `Site`/`Bond`/`Gate` data structures used by the rectangular grid. The color class assignment follows the hardware gate schedule (JSON layers 0, 1, 2). No changes are required in `gate.hpp`, `measures.hpp`, or `workspace.hpp` beyond parametrizing the workspace by `max_degree=3` instead of 4.

2.3 Hamiltonian and gate matrix

We simulate the isotropic Heisenberg (XXX) model:

$$H = J \sum_{\langle ij \rangle} \left(X_i X_j + Y_i Y_j + Z_i Z_j \right) \tag{1}$$

with uniform coupling $J = 1$ and time step $\delta t = 0.1$.

In the Heisenberg picture with Pauli basis, the two-site gate $U = e^{-i\theta(X\otimes X + Y\otimes Y + Z\otimes Z)}$ ($\theta = J\delta t$) is represented by a 16×16 real Pauli transfer matrix (PTM) with block structure:

$$\{II, XX, YY, ZZ\} \to \text{invariant (diagonal} = 1) \tag{2}$$
$$\{IX, XI, YZ, ZY\} \to 4 \times 4 \text{ mixing block} \tag{3}$$
$$\{IY, YI, XZ, ZX\} \to 4 \times 4 \text{ mixing block} \tag{4}$$
$$\{IZ, ZI, XY, YX\} \to 4 \times 4 \text{ mixing block} \tag{5}$$

with elements $\alpha = \cos^2(2\theta)$, $\beta = \sin^2(2\theta)$, $\gamma = \sin(2\theta)\cos(2\theta)$. This was verified against a Julia reference implementation.

3 Hardware Noise Model

We extend the simulation to include hardware noise channels characterized on the target device. In the Pauli transfer matrix (PTM) formalism, a noise channel $\mathcal{E}$ acting on a two-site operator is represented as a 16×16 real matrix $E_{ij} = d^{-1} \operatorname{Tr}(P_i^\dagger\, \mathcal{E}(P_j))$, where $\{P_i\}$ is the two-site Pauli basis. Since both operators and noise channels are Hermitian-preserving, E is real throughout, consistent with our float32 simulation.

Each bond implements the two-qubit interaction through a hardware decomposition $U_{\text{bond}} = R_{xx}(\alpha) \cdot R_{yy}(\alpha) \cdot R_{zz}(\delta)$, where $\alpha = -J_{\langle ij \rangle} \cdot \varepsilon$ scales with the bond coupling and Trotter step ε, while $\delta = -J_{\langle ij \rangle}$ is independent of ε. Noise is interleaved after each sub-gate, and the full noisy gate for bond b in its color class is:

$$\text{PTM}^{(b)}_{\text{noisy}} = G_1 \cdot M^{(b)}_n \cdot G_2(\alpha_b) \cdot M^{(b)}_n \cdot G_3(\alpha_b) \cdot M^{(b)}_n \cdot G_4(\delta_b) \tag{6}$$

where $M^{(b)}_n$ is the hardware noise PTM for bond b in its color class, and G_1–G_4 implement the hardware sub-layers. After all color classes, a kick gate $R_x(2\phi)$ is applied once per site ($\phi = 1.41371669$).

The noise characterization is fully parametric: a configuration file specifies an independent 16×16 PTM for each (bond, color class) pair and noise scaling factor $\gamma \in \{1.0, 2.0, 3.0\}$ ($\gamma = 1.0$: hardware noise as measured; $\gamma = 2.0, 3.0$: amplified for MPF extrapolation). This single composed PTM is pre-computed per bond at build time, so runtime applies exactly one matrix–vector product per bond per Trotter step — the same cost as the noiseless simulation.

4 Autocorrelation and Chi Convergence

We initialize the Heisenberg-picture operator as Z at a single center site (site 13, a degree-3 junction at the graph midpoint) and I elsewhere. The autocorrelation $C(t) = \langle Z_c(t) Z_c(0) \rangle$ is computed via the Bethe approximation [2, 8] after each Trotter step. We study convergence with respect to the bond dimension χ by running nine simulations with $\chi \in \{50, 100, 150, 200, 250, 300, 350, 400, 430\}$ over 30 Trotter steps at cutoff 10^{-4}.

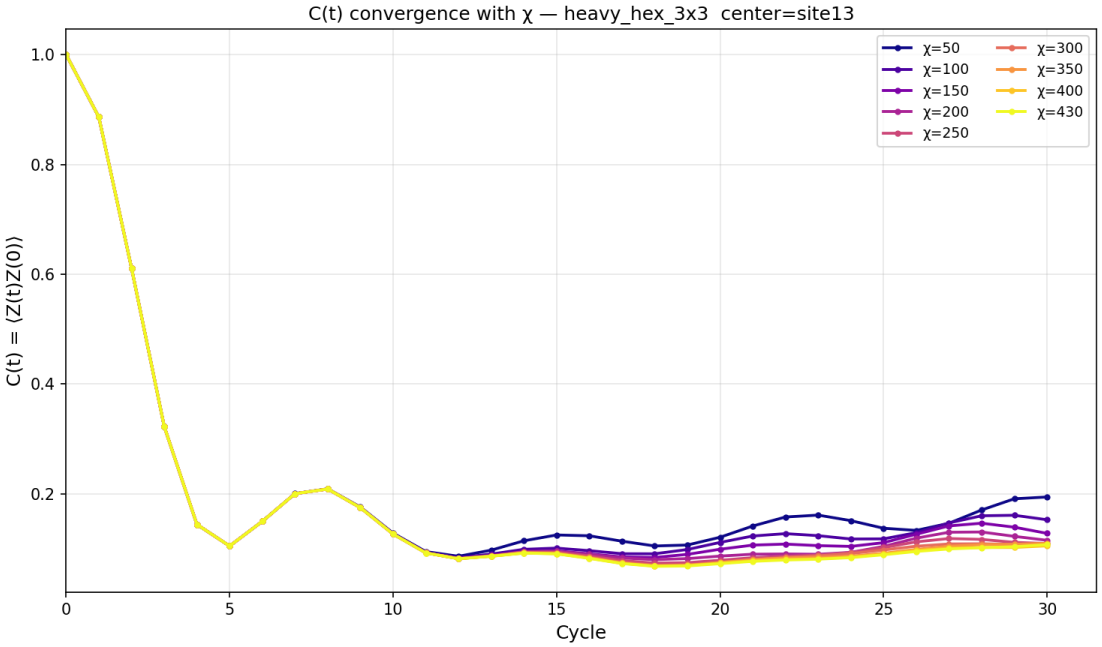


Figure 1: Autocorrelation $C(t) = \langle Z_c(t) Z_c(0) \rangle$ for $\chi = 50, 100, 150, 200, 250, 300, 350, 400, 430$ over 30 Trotter steps. The oscillatory behaviour is a genuine quantum recurrence of the XXX model on the finite heavy-hex graph. Curves for $\chi \geq 150$ are visually indistinguishable, indicating convergence.

Figure 1 shows $C(t)$ for $\chi = 50, 100, 150, 200, 250, 300, 350, 400, 430$ over 30 Trotter steps. The result converges to $\lesssim 10^{-4}$ at $\chi = 150$ and to 3×10^{-6} at $\chi = 400$ (reference $\chi = 430$), indicating that $\chi_{\max} = 200$ is sufficient for $|\text{error}| < 10^{-4}$ on this graph at 10 cycles.

The non-monotonic behaviour of $C(t)$ — a decrease followed by partial recovery — is a genuine quantum recurrence of the XXX model on the degree-3 lattice, confirmed by its independence from χ.

Figure 3 shows the non-identity Pauli weight $n(v,t)$ at each site for 20 cycles. The operator spreads radially outward from the center junction, following the 3-fold symmetry of the hexagonal lattice. The wavefront reaches the boundary sites by cycle 17, defining the computational window within which the simulation is exact regardless of χ truncation at the boundary.

5 Performance

We report wall-clock times on a 32 GB GPU using Householder QR decomposition (QR) and float32 arithmetic. All timings are per Trotter cycle at saturated bond dimension.

Table 1: Wall-clock time per cycle. GPU runs: χ capped at 430, float32, Householder QR, cutoff=10^{-4}. CPU: float64, no χ cap (grows freely to χ_{CPU}). Julia: GPU, noiseless evolution time only. '—': not measured.

Cycle	$\chi_{\max}$	CPU (s)	χ_{CPU}	Julia (s)	CppSim (s)	Speedup
2	14	9	14	29	0.1	290×
4	60	15	58	29	0.5	58×
6	165	201	202	38	1.9	20×
7	278	978	364	65	4.4	15×
8	430	4,393	637	132	12.3	11×
9	430	—	—	251	25.2	10×
10	430	—	—	418	46.9	9×

All runs saturate at their χ cap by cycle 7–9. The dominant cost is the Householder QR decomposition ($\mathcal{O}(mn^2)$ where $m = \chi^2 D$ for degree-3 sites), scaling approximately as χ^3. Table 1 compares wall-clock times at $\chi = 430$, cutoff=10^{-4}. Julia times report autocorrelation evolution only. At saturated bond dimension (cycles 8–10), CppSim achieves a consistent $\sim$10× speedup. At saturated bond dimension ($\chi = 430$, cycles 8–10), CppSim achieves a $\sim$10× speedup over the Julia/AMDGPU.jl reference implementation on the same GPU. Both use Householder QR and float32 arithmetic. The speedup arises from: (i) zero GPU memory allocation during simulation (pre-allocated workspace), (ii) direct HIP/rocBLAS calls without framework dispatch overhead, and (iii) on-device `pseudo_sqrt` kernel eliminating CPU–GPU transfers.

6 Multi-Product Formula Application

The gamma-MPF uses three noisy states at noise scaling factors $\gamma \in \{1.0, 2.0, 3.0\}$ with fixed Trotter step $\varepsilon = 0.1$, extrapolating to $\gamma = 0$ to recover the noise-free signal. The MPF extrapolation requires coefficients $\{c_i\}$ such that $C_{\text{mpf}}(t) = \sum_i c_i C_n(\gamma_i, t) \approx C_{\text{nl}}(t)$, with constraint $\sum_i c_i = 1$.

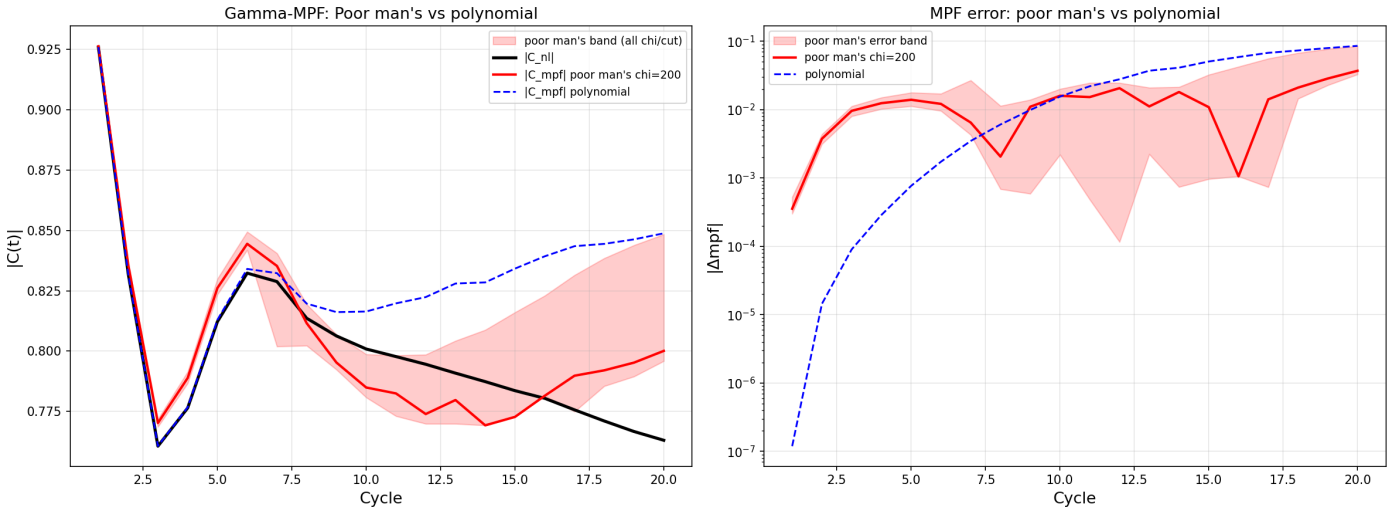


Figure 2: MPF error $|C_{\text{mpf}}(t) - C_{\text{nl}}(t)|$ (log scale) for gamma-MPF with $\chi = 200$, $\varepsilon = 0.1$, 20 Trotter cycles. Left: $|C_{\text{mpf}}(t)|$ vs $|C_{\text{nl}}|$ reference (dashed). Right: error comparison — poor man's MPF (red, correlation matrix) outperforms polynomial extrapolation (blue) by 2–3 orders of magnitude up to cycle 12. Band shows variation across all χ/cutoff values.

Rather than computing quantum inner products $\langle\psi_i|\psi_j\rangle$ (which requires solving a gauge-dependent cross-state contraction problem), we construct the coefficient matrix directly from the observable time series. We define the correlation matrix:

$$A_{ij} = \sum_t C_n(\gamma_i, t)\, C_n(\gamma_j, t), \qquad B_i = \sum_t C_n(\gamma_i, t)\, C_{\text{nl}}(t) \tag{7}$$

and solve $\min_{\mathbf{c}} \|A\mathbf{c} - \mathbf{B}\|^2$ subject to $\sum_i c_i = 1$. This *poor man's MPF* is gauge-invariant — it depends only on the autocorrelation $C(t)$, a physical observable — and requires no additional computation beyond the Trotter evolution already performed.

This formulation is also a natural scaffold for the full quantum inner product MPF: replacing A_{ij} with $\langle\psi_i|\psi_j\rangle$ recovers the complete formulation used in [6], which we leave for future work.

The polynomial extrapolation is a Taylor approximation in γ, valid when the noise perturbation is small. As cycles accumulate, the noise effect grows and higher-order terms become significant, causing the polynomial fit to diverge. The poor man's MPF (correlation matrix) exploits the full temporal structure of the signal and outperforms the polynomial by 2–3 orders of magnitude up to cycle 12 (Fig. 2). The full 10-cycle gamma-MPF computation at $\chi = 70$ completes in under 5 s on a 32 GB GPU.

7 Conclusion

We have demonstrated that CppSim, a C++/HIP tensor network simulator, extends to the heavy-hex lattice with minimal code changes. The topology-agnostic gate and BP kernels accept any `Grid`-derived

class; the heavy-hex extension requires only a new site/bond/color construction and a 16×16 Pauli transfer matrix for the XXX Hamiltonian.

On a 32 GB GPU, the 3×3 heavy-hex graph (68 sites, 76 bonds) is simulated with bond dimension $\chi \leq 430$. The autocorrelation $C(t)$ converges at $\chi = 200$ ($|\text{error}| < 10^{-4}$ relative to $\chi = 430$) for 10 Trotter cycles. The non-monotonic behaviour of $C(t)$ — a quantum recurrence of the isotropic Heisenberg model on the finite graph — is confirmed independent of χ and is therefore a physical prediction of the simulation. We demonstrate a $\sim$10$\times$ speedup over a Julia/AMDGPU.jl reference [5] at saturated bond dimension.

The operator lightcone, computed as a byproduct with no additional cost, visualizes operator spreading on the hexagonal lattice and defines the computational window within which the simulation is exact.

Future work will extend to larger graphs on higher-memory hardware, and implement interleaved BP to match the online approach of [2] and eliminate BP trap effects.

References

[1] C. Chamberland, G. Zhu, T. J. Yoder, J. B. Hertzberg, and A. W. Cross. Topological and subsystem codes on low-degree graphs with flag qubits. *PRX Quantum*, 2:030345, 2021.

[2] J. Tindall and M. Fishman. Gauging tensor networks with belief propagation. *SciPost Physics*, 15:222, 2023.

[3] J. Tindall, M. Fishman, E. M. Stoudenmire, and D. Sels. Efficient tensor network simulation of IBM's eagle kicked ising experiment. *PRX Quantum*, 5:010308, 2024.

[4] J. Tindall, A. Mello, M. Fishman, M. Stoudenmire, and D. Sels. Dynamics of disordered quantum systems with two- and three-dimensional tensor networks, 2025.

[5] M. S. Rudolph and J. Tindall. Simulating and sampling from quantum circuits with 2d tensor networks, 2025.

[6] G. H. Low, V. Kliuchnikov, and N. Wiebe. Well-conditioned multi-product formulas for hardware-friendly Hamiltonian simulation, 2019.

[7] Paolo D'Alberto. Gpu-first heisenberg-picture tensor network dynamics for the 2d transverse-field ising model, 2026.

[8] G. Evenbly, N. Pancotti, A. Milsted, J. Gray, and G. K.-L. Chan. Loop series expansions for tensor networks. *Physical Review Research*, 8:013245, 2026.

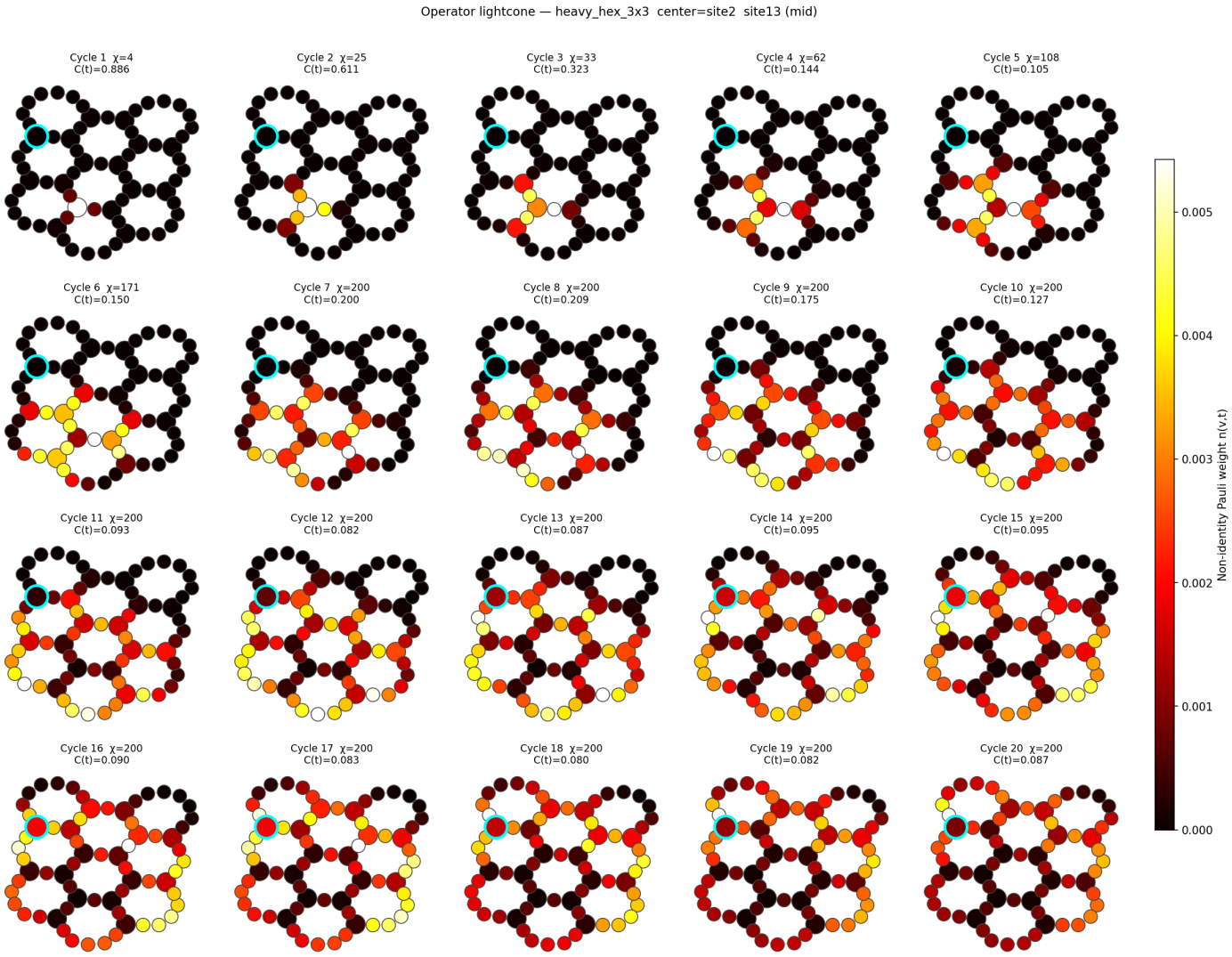


Figure 3: Non-identity Pauli weight $n(v,t)$ at each site of the heavy_hex_3x3 graph over 20 Trotter steps, initialised at site 13 (mid-graph degree-3 junction, marked cyan). The operator spreads radially outward with 3-fold hexagonal symmetry, reaching the graph boundary by cycle $\sim$17. Color scale is per-panel normalized.